

Jenkins pipelines : Intégration et Livraison Continues TP2

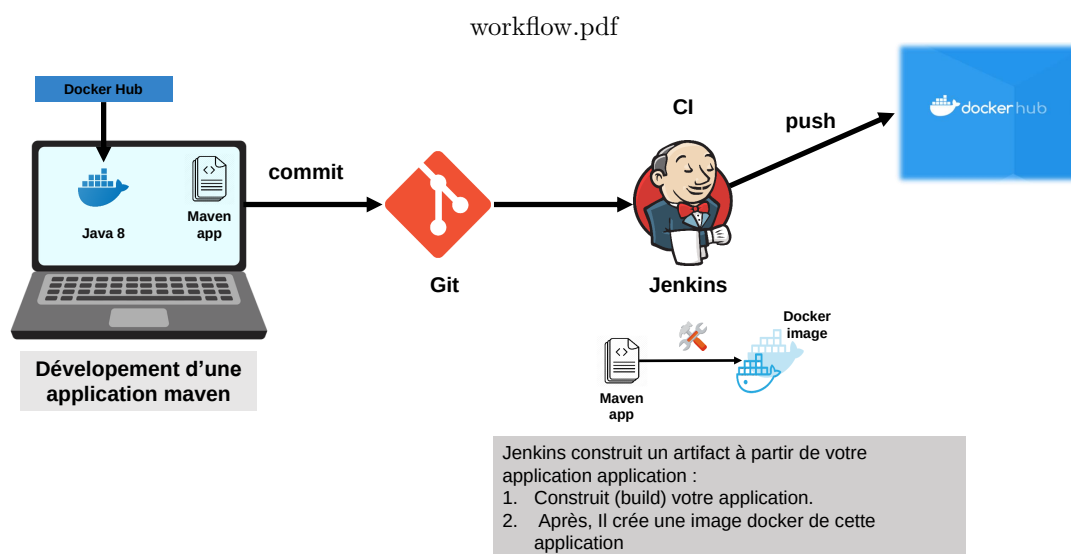
Notions Abordées :

- Intégration continue avec git Jenkins et Maven.
- pipeline , Jenkinsfile
- Jenkins et github webhook

Objectif

L'objectif de ce TP est d'utiliser le serveur jenkins pour compiler, tester et déployer régulièrement notre application. Nous allons suivre un processus complet de l'intégration continue.

Nous verrons particulièrement la notion de pipeline dans Jenkins.



Dans le cadre de ce travail, Nous allons utiliser un projet Maven et son script de compilation et tests, ainsi que son Dockerfile. Vous pouvez prendre une applications Java et le script Maven permettant de la compiler. Vous pouvez aussi prendre une application trouvée sur le net, du moment qu'elle soit accompagnée d'un script de compilation, des tests unitaire, et d'un Dockerfile.

(A) Pipeline comme un code

Rappel

Un pipeline Jenkins consiste en deux types d'éléments : *stages* et *steps*.

- *stage* : une séparation logique entre les étapes (steps), permettant ainsi de distinguer les séquences d'étapes, par exemple, Build, Unit Test, Deploy ... Ceci aidera à visualiser le progrès d'une pipeline Jenkins
- *step* : consiste en une opération, demandant à Jenkins la tâche à faire, par exemple, le check-out du code à partir d'un dépôt, exécuter un script, etc.

Pré-requis

- S'assurer que le serveur Jenkins a accès à Internet. (nécessaire pour télécharger et installer les plugins).
 - S'assurer que Jenkins dispose des plugins nécessaires pour le bon fonctionnement de votre code
- (a) Ajouter "Git Credentials" dans Jenkins Pour que Jenkins puisse communiquer avec GitHub, on a besoin d'ajouter, sur Jenkins, les identifiants (Credentials) du compte Github. (Assurez vous que Jenkins Credentials Plugin est installé).

Une façon de faire, pour ajouter les identifiants de github sur Jenkins, est de suivre les étapes suivantes :

- Placez vous sur "*Credentials > Users > Global Credentials*"
 - Dans le menu (à gauche), Cliquez sur Ajoutez Credentials (Identifiants)
 - Choisissez l'option **Username with password**
 - Ajoutez le **username et le mot de passe de votre compte GitHub**
 - Donnez à vos identifiants un mot de passe unique (exp: Github-Credentials)
 - Ajoutez une description (optionnel), et sauvegardez
- (b) "Global Tool Configuration
- Avant l'exécution de votre pipeline, tâchez à voir la page "Global Tools". C'est dans cette dernière que vous configurez des outils que vous pouvez utiliser tout au long de votre pipeline tel que : Java, Maven, Git
- Sur la section "*Manage Jenkins > Global Tools Configuration*", défiler vers le bas, jusqu'à arriver à la section Jenkins
 - Ajouter Maven, donnez lui un nom unique, et indiquez la version souhaitée.

De même pour Git.

Note : Si Jenkins ne reconnaît pas Maven ou Git, assurez vous également de l'installation d'un plugin Git et d'un plugin Maven ou Maven Integration Plugin.

Un Premier Exemple: Sur votre tableau de bord Jenkins, cliquez sur nouveau item → pipeline :

Jenkins > Tous >

Saisissez un nom

» Champ obligatoire

Construire un projet free-style

Ceci est la fonction principale de Jenkins qui sert à builder (construire) votre projet. Vous pouvez intégrer tous les outils de gestion de version avec tous les systèmes de build. Il est même possible d'utiliser Jenkins pour tout autre chose qu'un build logiciel.

Construire un projet maven

Construit un projet avec maven. Jenkins utilise directement vos fichiers POM et diminue radicalement l'effort de configuration. Cette fonctionnalité est encore en bêta mais elle est disponible afin d'obtenir vos retours.

Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

Construire un projet multi-configuration

Adapté aux projets qui nécessitent un grand nombre de configurations différentes, comme des environnements de test multiples, des binaires spécifiques à une plateforme, etc.

pour qui stocke des objets imbriqués. Utile pour grouper ensemble des éléments. Contrairement à une vue qui n'est qu'un

Si vous n'arrivez pas à trouver l'icône du pipeline, vous devrez d'abord installer le plugin pipeline.

Par exemple, soit une simple pipeline contenant deux "stages" :

Jenkins > First-pipeline >

General Build Triggers Advanced Project Options **Pipeline**

Pipeline

Definition Pipeline script

Script

```

1 pipeline{
2   agent any
3
4   stages{
5     stage('Premier Stage'){
6       steps{
7         echo 'Un premier Bonjour !'
8       }
9     }
10
11    stage('Deuxième Stage'){
12      steps{
13        echo 'Un deuxième Bonjour !'
14        echo 'Au revoir!'
15      }
16    }
17  }
18 }
19
20

```

☒ Use Groovy Sandbox

[Pipeline Syntax](#)

Sauvegardez votre pipeline et lancer un build. Une représentation visuelle devrait apparaître.

Stage View

	Premier Stage	Deuxième Stage
Average stage times:	815ms	552ms
(Average full run time: ~17s)		
#2 Oct 08 22:29 No Changes	815ms	552ms
#1 Oct 08 22:26 No Changes		

Liens permanents

- [Dernier build \(#2\), il y a 24 s](#)
- [Dernier build stable \(#2\), il y a 24 s](#)
- [Dernier build avec succès \(#2\), il y a 24 s](#)
- [Dernier build en échec \(#1\), il y a 3 mn 13 s](#)
- [Dernier build non réussi \(#1\), il y a 3 mn 13 s](#)

Jenkins vous permet de voir l'historique des builds, et vous pouvez aussi voir les détails d'exécution en cliquant sur le numéro du build et puis sur console. Si l'une des étapes échoue, le processus du build s'arrête, et aucune autre étape ne s'exécute. La pipeline nous permet aussi de visualiser le point d'échec.

Pipeline pour un projet Maven

1) Première étape : Avoir un projet avec des tests sur Github

Envoyez le code de votre projet accompagné des tests sur Github.

Rappel :

Initialiser votre dépôt avec `git init` Commitez votre code dans le repo, en ajoutant d'abord le code dans l'index.

- `git add .`
- `git commit -m "v1"`
- `git remote add origin votre_githubrepo`
- `git push -u origin master`

On peut créer maintenant une nouvelle pipeline nommée *pipeline-triangle*, entant qu'un script, et y mettra un premier "stage"

qui nous permettra de charger le code de notre projet à partir de github

Création d'un premier "stage"

qu'on nommera checkout, nous permettant de charger le code du projet à partir de github.

```

pipeline{
  agent any
  stages{
    stage('git checkout'){
      steps{
        git credentialsId: 'git_credentials', url: 'https://github.com/badre-09/Triang07.git'
      }
    }
  }
}

```

Le pipeline peut être exécuté sur n'importe quel agent, jusqu'ici notre pipeline ne contient qu'une seule étape (chargement du projet à partir de github). Après exécution, on devrait obtenir la vue suivante :

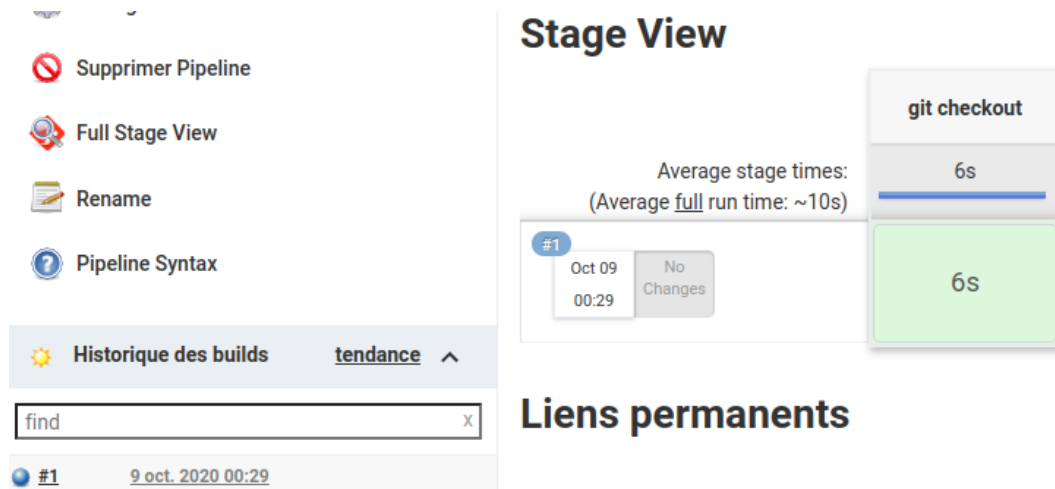


Figure 1:

2) Deuxième étape : Compilation de votre application

La deuxième étape du pipeline consiste en la compilation de l'application avec l'outil de build maven. **Notons, qu'il faut installer le plugin maven sur jenkins.**



Figure 2:

Sauvegardons le script du pipeline, et exécutons-le. La vue apparaît, mais cette fois-ci avec deux étapes :

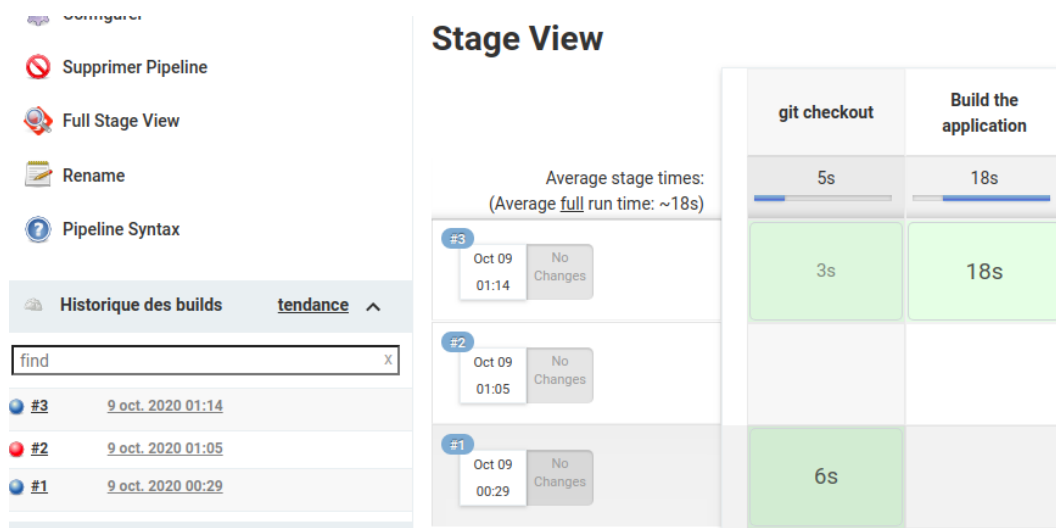


Figure 3:

Note ("Tools attribute") : Pour utiliser l'outil maven, on doit avoir cet outil pré-installé ou pré-configuré. Ainsi, Si le build génère une erreur par rapport à l'outil Maven, c'est qu'il n'arrive pas à le trouver. Dans ce cas, il faudra le définir dans l'attribut d'outils (tools attribute) dans votre script :

```

    tools{
Maven : Maven_nom_sur_Jenkins
    }
}

Maven "Maven_nom_sur_Jenkins"
Maven_nom_sur_Jenkins : le nom de l'outil pré-installé ou pré-configurer dans Jenkins résidant à :
manage>Jenkins>Global tool configuration>maven>name

```

3) Troisième étape : Exécution des test unitaires en ajoutant une autre étape à notre pipeline :

```

stage('Unit Test Execution') {
    steps{
        sh 'mvn test'
    }
}

```

4) Quatrième étape : Build de l'image docker

Notre projet devrait contenir un Dockerfile permettant de transformer notre application sous forme d'image Docker. Le build de l'image docker sera effectué à travers jenkins, en ajoutant au script de notre pipeline l'étape suivante :

```

stage('Build the docker image') {
    steps{
        sh "docker build -tag badre09/triang7:1.0.0. "
    }
}

```

5) Cinquième étape : Mettre l'image de le dépôt DockerHub

Pour ce, il faut d'abord se connecter à votre compte dockerhub.

Avant d'envoyer l'image docker à DockerHub, on va d'abord commencer par **définir les identifiants (Credentials)**.

Note : les identifiants sont d'abord définis dans l'interface graphique.

Vous aurez besoin du plugin : "Credentials Binding".

En utilisant la fonction withCredentials(..), Nous permet de passer le mot de passe en secret.

```

stage('Build the docker image') {
    steps{
        withCredentials([string(credentialsId: 'dockerhubpass', variable: 'dockerHubPass')]) {
            sh "docker login -u badre09-p $dockerHubPass "
        }
        sh 'docker push badre09/triang7:1.0.0'
    }
}

```

(B) Partie III : Pipeline avec Jenkinsfile

Le Jenkinsfile est un fichier à la racine du dépôt git. Il décrit l'exécution du pipeline avec la même syntaxe qu'un pipeline normal.

- Copiez le code de votre pipeline dans un fichier nommé Jenkinsfile.

- Poussez le fichier dans le git repository
- Sur Jenkins, configuration du pipeline, choisissez l'option *Pipeline script from SCM*

- Sur SCM, choisissez git et entrez l'adresse du dépôt contenant votre Jenkinsfile
- Sauvegarder et lancer votre build.

(C) Jenkins et Webhook

Dans ce TP, l'utilisation des webhooks nous permettra de déclencher automatiquement un build Jenkins à chaque fois qu'on réalise un commit sur github.

Pour que Jenkins puisse recevoir un trigger lui indiquant qu'un commit a été effectué sur github, il faut lui indiquer quel comportement suivre :

Placer vous dans la configuration de votre pipeline, dans la section *Build Triggers* puis cocher la case *"GitHub hook trigger for GITScm polling"*

Par la suite, placez-vous sur votre projet existant sur github repository, puis sur Configuration>webhook et ajouter un webhook.

Note : Pour pouvoir ajouter un webhook, il faut que votre adresse Jenkins soit public. Pour ce faire, on va transformer notre adresse local Jenkins en une adresse public en utilisant un logiciel appelé ngrok. Suivez les étapes suivantes :

- Téléchargez ngrok.zip
- Puis unzippez-le : `unzip /path/to/ngrok.zip`

- Exécutez cette instruction sur la ligne de commande :
`./ngrok authtoken 1if7mcXXzxHIimP3EjKOxS8ut7a_6b12Bn8Ghxdjfkf6CjgdzZ`
- Exécuter l'instruction : `./ngrok http 8080` (8080 étant mon Localhost)
- et enfin sur votre navigateur connectez vous avec le http ou le https généré.

```
ngrok by @inconshreveable

Session Status      online
Account             safau.achour@gmail.com (Plan: Free)
Version             2.3.35
Region              United States (us)
Web Interface        http://127.0.0.1:4040
Forwarding           http://43f332d1b16f.ngrok.io -> http://localhost:8080
Forwarding           https://43f332d1b16f.ngrok.io -> http://localhost:8080

Connections
  ttl    opn    rt1    rt5    p50    p90
   245    0     0.00   0.01   5.01   9.00

HTTP Requests
```

Effectuez une modification dans votre projet sur github pour vérifier si le build automatique sur Jenkins a été effectué

(D) **Post-action : notification par email**

- La première étape est de configurer votre serveur smtp pour envoyer un mail sur : manage Jenkins > configure system > extended email notification.
 Pour le cas de Gmail : smtp.gmail.com
 Port : 465
 cocher la case *SSL*
- Dans votre pipeline, on peut ajouter une post-action afin de pouvoir envoyer un email lors de l'échec d'un build

```
post{
    failure{
        email body: 'Ce Build $BUILD_NUMBER a échoué',
        recipientProviders:[requestor()], subject: 'build', to:
        'badre.bousalem@enpc.fr'
    }
}
```